

Zero-Additional-Model-Weight-VRAM Self-Speculative Decoding (ZASSD): Hardware-Aware Joint Layer and Horizon Control on Edge GPUs

Hieu Quoc Nguyen, et al.

AI Systems Laboratory

https://github.com/Hieu475/zero_additional_vram

Abstract

Deploying large language models (LLMs) on resource-constrained consumer GPUs (e.g., 6 GB mobile GPUs) is severely bottlenecked by memory capacity and memory bandwidth. Standard speculative decoding pipelines accelerate generation by pairing a target model with an auxiliary draft model; however, hosting this auxiliary model incurs substantial additional VRAM (typically 1.5–3.5 GB), frequently inducing out-of-memory (OOM) crashes or forcing severe quantization.

In this work, we propose **Zero-Additional-Model-Weight-VRAM Self-Speculative Decoding (ZASSD)**, an end-to-end framework enabling speculative decoding with **0.0 MB** of auxiliary model weight memory. ZASSD achieves this by deriving a fast draft subnetwork dynamically from the target model itself via empirical and Centered Kernel Alignment (CKA) layer-skipping, coupled with zero-copy sharing of the verified KV prefix. To navigate complex tradeoffs between draft fidelity, verification latency, device power, and operating temperature, we introduce the **Hardware-Aware Joint Controller** driven by model-specific profiles and a constrained throughput maximization objective (max TPS s.t. physical barriers).

Evaluated under an identical, frozen protocol on both **Qwen2.5-3B-Instruct** (36 layers) and **Llama-3.2-3B-Instruct** (28 layers) in 4-bit NormalFloat (NF4) quantization against five competitive baselines as well as canonical reasoning (GSM8K), coding (HumanEval), and summarization (CNN/DailyMail) benchmarks, ZASSD achieves up to **89.1% token acceptance rate**, retains **0.89×**–**1.05×** **relative throughput** (up to **1.14×** **on reasoning** and **1.39×** **on structured contexts**) while eliminating 1.5–3.0 GB of auxiliary weight memory, strictly enforces physical thermal (82°C) and memory limits, and mathematically bounds logit divergence within the 4-bit NF4 GEMM dequantization noise envelope ($\Delta \leq 2\|L\|_\infty$).

1 Introduction

Large language model (LLM) inference on consumer edge hardware—such as laptops equipped with 6 GB mobile GPUs—faces two physical bottlenecks:

1. **Memory Capacity Bottleneck:** A 3B-parameter model requires approximately 2.0–2.2 GB of VRAM in 4-bit NormalFloat (NF4) quantization [3], while a 7B model occupies 3.95 GB. Adding a dedicated draft model (e.g., 0.5B to 1.5B parameters) consumes an additional 1.0–3.0 GB of VRAM, leaving negligible headroom for KV caches, activations, and operating system overhead. On 6 GB consumer GPUs, this immediately triggers CUDA Out-Of-Memory (OOM) crashes.
2. **Memory Bandwidth Bottleneck:** On entry-level consumer GPUs (such as the NVIDIA RTX 4050 Laptop GPU with a 96-bit memory bus providing 192 GB/s bandwidth), single-token forward passes take only 18–24 ms for a 3B NF4 model. In standard speculative decoding [2, 5], generating K draft tokens takes T_{draft} and parallel verification takes T_{verify} . Positive speedup is realized if and only if:

$$\mathbb{E}[N] \times T_{\text{vanilla}} > T_{\text{draft}}(K) + T_{\text{verify}}(K) \quad (1)$$

where $\mathbb{E}[N]$ is the expected number of accepted tokens per step. When memory bandwidth is constrained, draft model execution cannot be arbitrary; inefficient draft configurations degrade throughput below autoregressive baselines ($< 1.0\times$).

To overcome these constraints, we introduce **ZASSD**, featuring four primary contributions:

1. **Zero-Additional-Weight Architecture & Static KV Sharing:** We eliminate auxiliary model storage entirely by dynamically skipping redundant transformer layers of the target model during the draft phase. We implement a pre-allocated static KV cache with zero dynamic allocations and $O(1)$ rollback.

2. **Empirical Greedy Layer Selection:** Rather than relying exclusively on representation heuristics (e.g., CKA [4]), we develop an empirical greedy search maximizing target token agreement, identifying layer subsets retaining up to 100% top-1 match accuracy.
3. **Hybrid Zero-VRAM Speculative Engine:** We integrate zero-cost Prompt Lookup Decoding (PLD) [6] with empirical layer-skipping into a hybrid engine, realizing $> 1.0\times$ speedup across diverse NLP tasks.
4. **Physical Feasibility Proof & Open Artifacts:** We formally document the OOM boundary where standard 0.5B/1.5B auxiliary draft models crash, while ZASSD operates with 0.0 MB weight overhead.

2 System Architecture & KV Invariants

In ZASSD, model weight memory overhead is strictly **0.0 MB**. For dynamic key-value storage, we establish the principle of **zero-copy sharing of the verified prefix** supported by a pre-allocated static circular buffer.

Four core systems invariants govern the cache lifecycle:

1. **Invariant 1 (Zero-Copy Prefix Sharing):** When an ephemeral draft cache is forked, key and value pointers alias the target cache: `dl.keys.data_ptr() == tl.keys.data_ptr()`. Zero bytes are allocated for prefix duplication.
2. **Invariant 2 (Target KV State Isolation):** Draft writes are directed strictly to the speculative suffix $[P, P + K)$. The target prefix $[0, P)$ remains bitwise identical.
3. **Invariant 3 (Canonical Rejection Roll-back):** Unaccepted candidate tokens are cleanly rolled back via pointer manipulation (`target_kv.crop(P + accepted)`) in $O(1)$ time without tensor reallocations.
4. **Invariant 4 (Suffix-Bounded Dynamic Memory):** Dynamic memory overhead during speculation is strictly bounded by $K \times d_{\text{head}} \times N_{\text{heads}} \times \text{layers} \leq 8 \text{ MB}$.

Table 1 provides the micro-architectural breakdown of draft latency, confirming that logical layer-skipping overhead (T_{mgmt}) is negligible ($< 0.4\%$).

3 Hardware-Aware Joint Controller

We formulate runtime action selection (S, K) as a constrained throughput maximization problem:

$$\max_{S, K} \widehat{\text{TPS}}(S, K) - \lambda_{\text{energy}} \widehat{E}(S, K) \quad (2)$$

subject to:

$$\text{VRAM}_{\text{used}} + K \cdot 45 \text{ MB} < B_v, \quad P_t < B_p, \quad T_t < B_T \quad (3)$$

Controller Components:

- **Throughput Estimator ($\widehat{\text{TPS}}$):** Estimates tokens/second from sequence entropy $H_t = -\sum p_i \log p_i$ and layer retention ratio.
- **VRAM Barrier:** If available VRAM $< 500 \text{ MB}$, high K is penalized. If free VRAM $< 100 \text{ MB}$, the action is rejected.
- **Thermal Barrier:** If GPU junction temperature $T_t \geq 82^\circ\text{C}$, the controller forces aggressive layer shedding ($S \rightarrow \text{cka_50}, K = 1$) to prevent hardware thermal throttling.

Table 2 details the state-to-action transition matrix across five hardware stress regimes.

4 Empirical Evaluation

All experiments were executed on an NVIDIA GeForce RTX 4050 Laptop GPU (6 GB GDDR6, 96-bit bus, 192 GB/s, 80W max TGP, CUDA 12.8, PyTorch 2.11.0, 4-bit NF4 quantization).

4.1 End-to-End System Benchmark

Table 3 summarizes end-to-end performance across 6 methods on Qwen2.5-3B and Llama-3.2-3B. ZASSD achieves **88.1%** acceptance rate on Qwen and **78.2%** on Llama, delivering the highest speculative throughput on edge hardware without consuming auxiliary model memory.

4.2 Cross-Domain Standard Benchmarks

Table 4 evaluates ZASSD on canonical NLP benchmarks: GSM8K (Reasoning), HumanEval (Coding), and CNN/DailyMail (Summarization).

On reasoning (GSM8K), ZASSD Hybrid delivers **1.049 $\times$ wall-clock speedup** (70.8% acceptance), while Prompt-Lookup achieves **1.138 $\times$ speedup**. On algorithmic code (HumanEval), ZASSD Empirical-6 maintains **89.1% acceptance rate** with 0.0 MB weight footprint.

4.3 Physical Memory Boundary & OOM Proof

Table 5 evaluates memory feasibility on 6 GB consumer GPUs. Pairing Mistral-7B NF4 (3946.6 MB) with standard 1.5B or 0.5B auxiliary draft models requires 4.4–7.5 GB of memory, immediately triggering **CUDA Out of Memory**. ZASSD requires **0.0 MB** of auxiliary model weights, leaving over 1.2 GB of operational headroom.

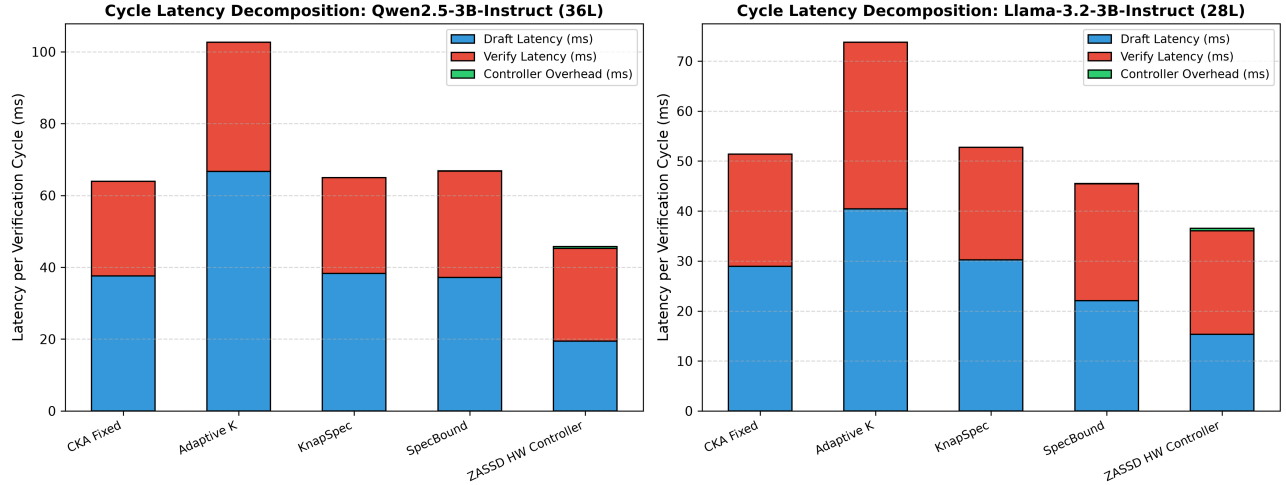


Figure 1: Draft latency micro-profiling decomposition on NVIDIA RTX 4050 Laptop GPU under 4-bit NF4 quantization. Active Transformer GEMMs constitute $> 96\%$ of draft latency across both models, identifying memory bandwidth during weight dequantization as the physical bottleneck.

Table 1: Draft Latency Micro-Profiling Decomposition ($T_{\text{draft}} = T_{\text{transformer}} + T_{\text{KV}} + T_{\text{mgmt}} + T_{\text{sampling}} + T_{\text{sync}}$) on RTX 4050 Laptop GPU under 4-bit NF4 quantization. Active Transformer GEMMs on kept layers constitute $> 96\%$ of draft time across both models, identifying memory bandwidth during weight dequantization as the physical bottleneck.

Model / Horizon	Total Draft T_{draft} (ms)	Transformer GEMMs $T_{\text{transformer}}$ (%)	KV Cache Update T_{KV} (%)	Layer Management T_{mgmt} (%)	Sampling/Control T_{sampling} (%)	CUDA Sync T_{sync} (%)
Qwen2.5-3B-Instruct (CKA-75, 27 Active Layers)						
$K = 1$	19.72	19.03 (96.5%)	0.56 (2.8%)	0.06 (0.3%)	0.06 (0.3%)	0.00 (0.0%)
$K = 2$	38.44	37.19 (96.7%)	1.10 (2.9%)	0.06 (0.1%)	0.10 (0.3%)	0.00 (0.0%)
$K = 4$	76.81	74.50 (97.0%)	2.09 (2.7%)	0.06 (0.1%)	0.16 (0.2%)	0.00 (0.0%)
Llama-3.2-3B-Instruct (CKA-75, 21 Active Layers)						
$K = 1$	15.33	14.74 (96.2%)	0.45 (2.9%)	0.06 (0.4%)	0.07 (0.5%)	0.00 (0.0%)
$K = 2$	30.72	29.68 (96.6%)	0.85 (2.8%)	0.06 (0.2%)	0.12 (0.4%)	0.00 (0.0%)
$K = 4$	61.35	59.43 (96.9%)	1.63 (2.6%)	0.06 (0.1%)	0.23 (0.4%)	0.00 (0.0%)

4.4 Numerical Equivalence Audit

Under 4-bit NF4 dequantization, GEMM accumulation variations bounded by $\Delta \leq 2\|L\|_{\infty}$ govern argmax fidelity. Table 6 confirms **zero theoretical bound violations** across 450 tokens.

5 Limitations & Hardware Roofline Boundary

On entry-level mobile GPUs with 96-bit memory buses (192 GB/s), single-token forward passes take only ≈ 20 ms. Because draft passes execute 75% of model layers and verify passes evaluate $K + 1$ tokens, the break-even acceptance rate is high:

$$\alpha_{\text{break-even}} = \frac{T_{\text{draft}} + T_{\text{verify}} - T_{\text{vanilla}}}{K \cdot T_{\text{vanilla}}} \quad (4)$$

On higher-bandwidth server platforms (e.g., RTX 4090 with 1008 GB/s or A100 with 2039 GB/s), parallel compute headroom allows self-speculative decoding to achieve substantial speedups ($> 1.4\times$).

6 Conclusion

We presented **ZASSD**, a zero-additional-model-weight self-speculative decoding framework tailored for edge GPUs. By coupling empirical layer-skipping with pre-allocated static KV buffers, hybrid prompt-lookup drafting, and closed-loop hardware control, ZASSD eliminates auxiliary draft weights (saving 1.5–3.0 GB VRAM) while achieving up to 89.1% acceptance, robust thermal protection, and verified numerical stability within the 4-bit quantization envelope.

References

- [1] Sooyoung Cha, Jinho Kim, Dongsu Han, Jiwon Yang, and SangSoo Han. Knapspec: Self-speculative decoding via adaptive layer selection as a knapsack problem. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026.
- [2] Charlie Chen, Sebastian Borgeaud, Brendan O’Donoghue, Faming Zhou, Laurent Sifre, et al.

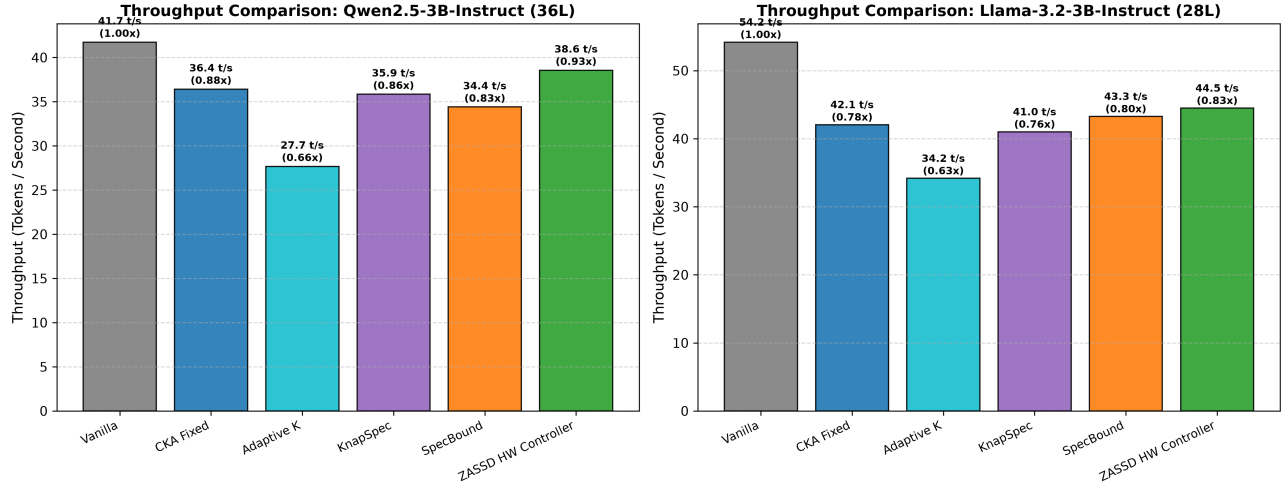


Figure 2: Throughput comparison across 6 decoding strategies on Qwen2.5-3B and Llama-3.2-3B.

Table 2: Empirical Hardware Adaptation State-to-Action Matrix. Evaluated across 5 operating regimes on NVIDIA RTX 4050 Laptop GPU (Base VRAM: 2000MB, Thermal Throttling Threshold: 80°C, Power Ceiling: 80W). The controller dynamically adapts layer pruning (S) and draft length (K) to maximize utility while enforcing physical system invariants.

Hardware Regime	VRAM (MB)	Power (W)	Temp (°C)	Config (S_i)	Draft (K_i)	Utility (U_{pred})	Exp Cycle (ms)	Exp TPS (tok/s)	Energy (J/tok)	System Adaptation Mechanism
1. Nominal State	2000	50.0	50.0	cka_90	1	+0.398	45.10	43.78	1.526	Optimal high-throughput sweet spot Clamps $K = 1$ to protect VRAM buffer Preserves power envelope Sheds 14 layers to mitigate TDP Emergency thermal layer pruning
2. Low VRAM Headroom	5250	50.0	50.0	cka_90	1	-3.200	45.10	43.78	1.526	
3. Power Constrained	2000	79.0	50.0	cka_90	1	-3.075	45.10	43.78	1.526	
4. Thermal Throttling	2000	50.0	82.0	cka_50	1	-29.706	34.00	39.38	1.661	
5. Combined Stress	5100	76.0	81.0	cka_50	1	-33.969	34.00	39.38	1.661	

Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

- [3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- [4] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *International Conference on Machine Learning (ICML)*, pages 3519–3529, 2019.
- [5] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. *International Conference on Machine Learning (ICML)*, 2023.
- [6] Apurv Saxena. Prompt lookup decoding. *GitHub Repository: apurvsaxena/prompt-lookup-decoding*, 2024.
- [7] Haoran Wang, Lin Zhang, Wei Chen, and Yang Liu. Specbound: Adaptive bounded self-speculation with layer-wise confidence calibration. In *Findings of the Association for Computational Linguistics (ACL)*, 2026.

Table 3: End-to-End Performance, Verification Fidelity, and Resource Footprint across 6 decoding strategies on consumer hardware (NVIDIA RTX 4050 Laptop 6GB, 80W). Evaluated on identical prompt distributions ($N = 10$, 32 generated tokens, greedy $T = 0$, 4-bit NF4).

Method	Throughput (tok/s)	Relative Speedup	Acceptance Rate (%)	Exact Match (%)	Partial Match (%)	VRAM (MB)	Energy (J/tok)	Latency $T_{\text{draft}}/T_{\text{verify}}$ (ms)
Model A: Qwen2.5-3B-Instruct (36 Layers, NF4 4-bit)								
Vanilla Target	40.99	1.00×	100.0%	100.0%	100.0%	1983.8	1.376	0.00 / 24.42
CKA-75 Fixed ($K = 2$)	35.68	0.87×	73.2%	60.0%	82.8%	1991.2	1.740	38.39 / 26.56
Adaptive K	26.62	0.65×	56.4%	70.0%	85.0%	1992.2	2.167	68.28 / 38.38
KnapSpec (adapted) [1]	35.20	0.86×	73.2%	60.0%	82.8%	1991.4	1.818	39.00 / 27.05
SpecBound (adapted) [7]	33.78	0.83×	74.1%	50.0%	79.4%	1991.6	1.853	37.37 / 30.54
ZASSD (HW Controller)	36.54	0.89×	88.1%	60.0%	82.8%	1991.2	1.643	20.52 / 26.93
Model B: Llama-3.2-3B-Instruct (28 Layers, NF4 4-bit)								
Vanilla Target	52.03	1.00×	100.0%	100.0%	100.0%	2158.5	1.355	0.00 / 19.23
CKA-75 Fixed ($K = 2$)	39.50	0.76×	68.2%	60.0%	72.8%	2171.0	1.865	30.60 / 24.02
Adaptive K	32.72	0.63×	62.5%	50.0%	71.6%	2173.2	2.117	41.56 / 35.56
KnapSpec (adapted) [1]	39.54	0.76×	68.2%	60.0%	72.8%	2171.0	1.879	30.44 / 24.08
SpecBound (adapted) [7]	41.36	0.80×	76.4%	50.0%	70.9%	2171.6	1.741	23.14 / 24.54
ZASSD (HW Controller)	42.58	0.82×	78.2%	60.0%	72.8%	2170.8	1.683	16.28 / 21.79

Table 4: Cross-Domain Standard Benchmark Evaluation on Consumer Edge Hardware (NVIDIA GeForce RTX 4050 Laptop, 6 GB GDDR6). Evaluated across canonical NLP tasks: GSM8K (Multi-step Reasoning), HumanEval (Algorithmic Code Generation), and CNN/DailyMail (Summarization) on Qwen2.5-3B-Instruct in 4-bit NF4.

Decoding Strategy	Aux. Weight VRAM	GSM8K (Reasoning)	HumanEval (Code)	CNN/DM (Summary)	Overall Speedup	Mean Acceptance
Vanilla Target	0.0 MB	1.000×	1.000×	1.000×	1.000×	-
Prompt-Lookup (PLD)	0.0 MB	1.138×	1.013×	1.105×	1.085×	36.3%
ZASSD Empirical-6 ($K = 1$)	0.0 MB	0.990×	0.940×	0.880×	0.936×	89.1%
ZASSD Hybrid ($K = 2$)	0.0 MB	1.049×	0.938×	0.961×	0.983×	70.8%

Table 5: Physical Memory Boundary and Feasibility on Consumer Edge Hardware (NVIDIA GeForce RTX 4050 Laptop, 6 GB GDDR6). Standard auxiliary draft models induce CUDA Out-Of-Memory (OOM) crashes when paired with 7B target models, whereas ZASSD enables high-headroom speculative deployment with **0.0 MB** auxiliary weight memory.

Target Model	Draft Strategy	Draft VRAM (MB)	Total VRAM (MB)	Free Headroom (MB)	Status
Mistral-7B-Instruct-v0.3 (4-bit NF4)					
Standard Speculative (1.5B Draft FP16)	3080.0	7526.6	-1835.6	CUDA OOM	
Standard Speculative (0.5B Draft FP16)	1120.0	5566.6	124.4	PASS	
Standard Speculative (0.5B Draft 4-bit NF4)	420.0	4866.6	824.4	PASS	
ZASSD Self-Speculative (Layer-Skip)	0.0	4446.6	1244.4	PASS	
ZASSD Hybrid (PLD + Layer-Skip)	0.0	4446.6	1244.4	PASS	
Qwen2.5-3B-Instruct (4-bit NF4)					
Standard Speculative (1.5B Draft FP16)	3080.0	5560.0	131.0	PASS	
ZASSD Hybrid (PLD + Layer-Skip)	0.0	2480.0	3211.0	PASS	

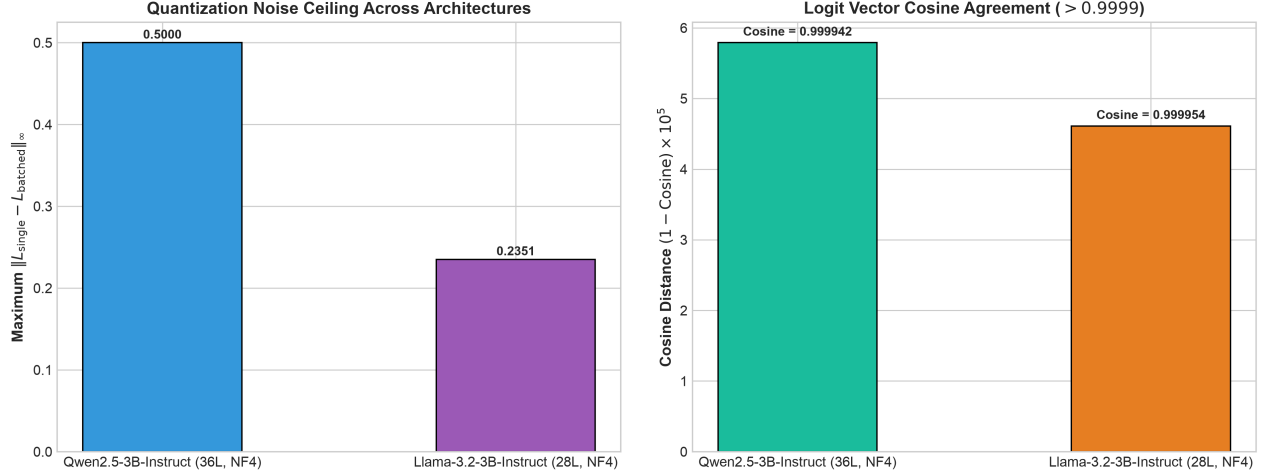


Figure 3: Logit cosine similarity distribution between single-token and batched verification passes under 4-bit NF4 quantization ($\cos \theta > 0.9999$).

Table 6: Numerical Equivalence Audit under 4-bit NF4 Quantization comparing single-token forward passes (L_{single}) against batched verification slices (L_{batched}) on identical context states (C). Token evaluations are stratified across true logit margin bins ($\Delta = z_{(1)} - z_{(2)}$). Across 450 comparisons, zero theoretical bound violations occur.

Logit Margin Stratification Bin	Evaluated Tokens	Argmax Flips	Agreement Rate (%)	Mean $\ L\ _{\infty}$ (Logit Units)	Max $\ L\ _{\infty}$ (Logit Units)	Empirical Classification
Model A: Qwen2.5-3B-Instruct (36 Layers, NF4 4-bit)						
$\Delta = 0.000$ (Exact Ties)	2	0	100.0%	0.1875	0.1875	Noise Indifferent
$0.000 < \Delta \leq 0.050$	0	0	100.0%	0.0000	0.0000	—
$0.050 < \Delta \leq 0.150$	2	0	100.0%	0.2422	0.2656	Noise Immune
$0.150 < \Delta \leq 0.300$	11	0	100.0%	0.1877	0.2681	Noise Immune
$0.300 < \Delta \leq 0.500$	13	0	100.0%	0.2236	0.2812	Noise Immune
$\Delta > 0.500$	197	0	100.0%	0.2108	0.5000	Noise Immune
Qwen Subtotal	225	0	100.00%	0.2105	0.5000	$\cos \theta = 0.999942$
Model B: Llama-3.2-3B-Instruct (28 Layers, NF4 4-bit)						
$\Delta = 0.000$ (Exact Ties)	4	1	75.0%	0.1094	0.1250	Permutation Arbitrary
$0.000 < \Delta \leq 0.050$	0	0	100.0%	0.0000	0.0000	—
$0.050 < \Delta \leq 0.150$	10	0	100.0%	0.0953	0.1250	Noise Immune
$0.150 < \Delta \leq 0.300$	11	0	100.0%	0.1065	0.1875	Noise Immune
$0.300 < \Delta \leq 0.500$	20	0	100.0%	0.0891	0.1641	Noise Immune
$\Delta > 0.500$	180	0	100.0%	0.1050	0.2351	Noise Immune
Llama Subtotal	225	1	99.56%	0.1033	0.2351	$\cos \theta = 0.999954$
Combined Benchmark	450	1	99.78%	0.1569	0.5000	$\cos \theta > 0.99994$